Rapport projet C++

Augustin BERNARD - Coralie TOSTEN - Bryan LEVY - Julien BOUSSET

Introduction **2**
Organisation du code **2**
Stratégie **3**
- Stratégie globale 3
- Les différentes fonctions / structures composant notre stratégie 3
  - fonction probabilite_victoire 3
  - fonction PlayerCells 3
  - fonction IdentifyCluster 3
  - structure Connection 3
  - fonction SCellToConnect 4
  - fonction WidthPath 4
  - structure ResultatChemin 4
  - fonction AStar 4
  - fonction _AStar 4
  - fonction ReunirTerritoire 5
  - fonction AttackOrNot 5

Génération de carte **6**
- Analyse 6
- Modèle Objet 7
- Principe de la génération 8
- Paramètres de l'algorithme 9

Conclusion **10**

Introduction

L'équipe pour ce projet en c++ est composée d'Augustin BERNARD, Coralie TOSTEN, Bryan LEVY et Julien BOUSSET.

Augustin s'est occupé de la conception et de l'implémentation de l'algorithme original de génération de la carte.

Bryan, Coralie et Julien ont collaboré sur l'intelligence artificielle de la stratégie de jeu.

- Bryan a géré les aspects annexes et d'intégration :
    - Développement du script pour faire fonctionner l'interface web sur Firefox.
    - Implémentation de la stratégie dummy
    - Mise en commun et intégration des différentes parties du code.
- Coralie a été responsable des fonctions de base et d'analyse structurelle :
    - PlayerCells => Ensemble des cellules du joueur
    - IdentifyCluster : => Ensemble des clusters des cellules du joueur
    - SCellToConnect => Les cellules desquels partir pour relier les clusters
    - WidthPath => Parcours en largeur
- Julien a développé les algorithmes de décision et de planification avancés :
    - probabilite_victoire => Revoie les probabilité entre deux nombre de dés
    - AStar et _AStar => Trouve un chemin pour réunir les clusters
    - ReunirTerritoire => À partir du chemin réunit les clusters
    - AttackOrNot => défini si on doit arrêter d'attaquer à ce tour

Organisation du code

L'ensemble du code de notre stratégie est codé dans le répertoire 'src/Projects/StrategyTest/'

Le code de la génération de carte se situe dans le répertoire 'src/Projects/GenMap/src/' sur les fichiers RandomMap.cpp et RandomMap.h

Stratégie

Stratégie globale

Dans un premier temps, on regarde si nos cellules sont regroupées dans un unique cluster à l'aide de IdentifyCluster, afin de gagner plus de dés lors du renforcement.
Si ce n'est pas le cas :

On cherche à les regrouper à l'aide de ScellToConnect qui retourne les deux meilleures cellules à relier, toujours dans le but de gagner plus de dés lors du renforcement. Ces deux cellules sont données à l'algorithme AStar afin de trouver un chemin pour les relier. Ce chemin est ensuite donné à ReunirTerritoire afin de les relier de la façon la plus sûre.

Si c'est le cas :

On utilise AttackOrNot afin de savoir si on peut attaquer à ce tour. Si on peut attaquer sans danger à ce tour alors on va parcourir toutes nos cellules afin d'attaquer à l'endroit ayant le plus de chance de réussite.

Les différentes fonctions / structures composant notre stratégie

Notre stratégie utilise plusieurs fonctions et structure pour pouvoir prendre des décisions:

fonction probabilite_victoire

Entrées : deux int représentant le nombre de dés possédés par 2 territoires
Sortie : un float représentant une probabilité de victoire entre 0% et 100%

Cette fonction permet de connaître le pourcentage de chances de victoire en fonction du nombre de dés présents. Les probabilités sont écrites en dure car le calcul pour un nombre de dés important était trop long.

fonction PlayerCells

Entrées : la carte et l'identifiant du joueur
Sortie : les cellules qui appartiennent au joueur

fonction IdentifyCluster

Entrées : les cellules appartenant au joueur (PlayerCells) et l'identifiant du joueur
Sortie : les clusters des cellules appartenant au joueur

structure Connection

Cette structure va servir pour identifier les cellules à rejoindre. Elle comporte deux cellules : un départ et une arrivée ainsi que la distance qui les sépare.

fonction SCellToConnect

Entrées : la carte et l'identifiant du joueur
Sortie : une connection
Cette fonction renvoie les 2 cellules les plus favorables à rejoindre selon les critères de distance, de nombre de dé et si elles sont dans des grands clusters.
Si toutes les cellules sont dans un même cluster, la fonction renverra NULL
Elle utilise la fonction IdentifyCluster pour calculer les clusters ainsi que la fonction WidthPath qui sert à calculer les distances entre les cellules.

fonction WidthPath

Entrée : la carte et une cellule
Sortie : le parcours en largeur en partant de la cellule.
Le parcours est sous forme d'une liste de couches: chaque couche correspond aux voisins à des distances différentes. Par exemple, la couche 1 va contenir les voisins directs de la cellule, la couche 2 les cellules à une distance de 1 de la cellule.

structure ResultatChemin

Cette structure va servir pour stocker le chemin qui servira à relier deux cellules de cluster différent afin d'avoir plus de dés lors du renforcement et la probabilité de compléter ce chemin (en n'utilisant que les dés du point de départ).

fonction AStar

Entrées : la carte, une cellule de départ et une cellule d'arrivée.
Sortie : la structure ResultatChemin obtenue grâce à _AStar

Cette fonction permet d'appeler la fonction récursive _AStar et récupère le meilleur chemin créé par elle.

fonction _AStar

Entrées : la cellule actuelle (celle de départ lors du premier appel), la cellule d'arrivée, le chemin actuel, la probabilité actuelle, le nombre de dés restant, les id des cellules déjà visités
Sortie : rien, elle met à jour une variable globale durant son exécution.

Cette fonction parcourt tous les chemins possibles (de longueur inférieur au nombre de dés de départ) afin de trouver le meilleur marinière de réunir les 2 cellules.
On utilise les id des cellules déjà visitées pour éviter de faire des cycles. A chaque tour le nombre de dés restant est décrété car il est obligatoire de laisser un dés sur les cellules conquises.
Si nous avions eu plus de temps nous aurions pu faire un algorithme qui permet de calculer le chemin en prenant en compte que l'on peut aussi utiliser le point d'arrivée pour faire rejoindre les deux cellules.

fonction ReunirTerritoire

entrées : Résultat chemin, la carte, le choix de l'action à faire lors du tour
sortie : un booléen : true = on a sélectionné une action, false = on peut rien faire il faut supprimer le chemin
A partir du chemin donné l'algorithme va essayer de réunir les territoires en commençant par attaquer là où il a le plus de chance jusqu'à réussir ou ne plus avoir de possibilité de réussir.

fonction AttackOrNot

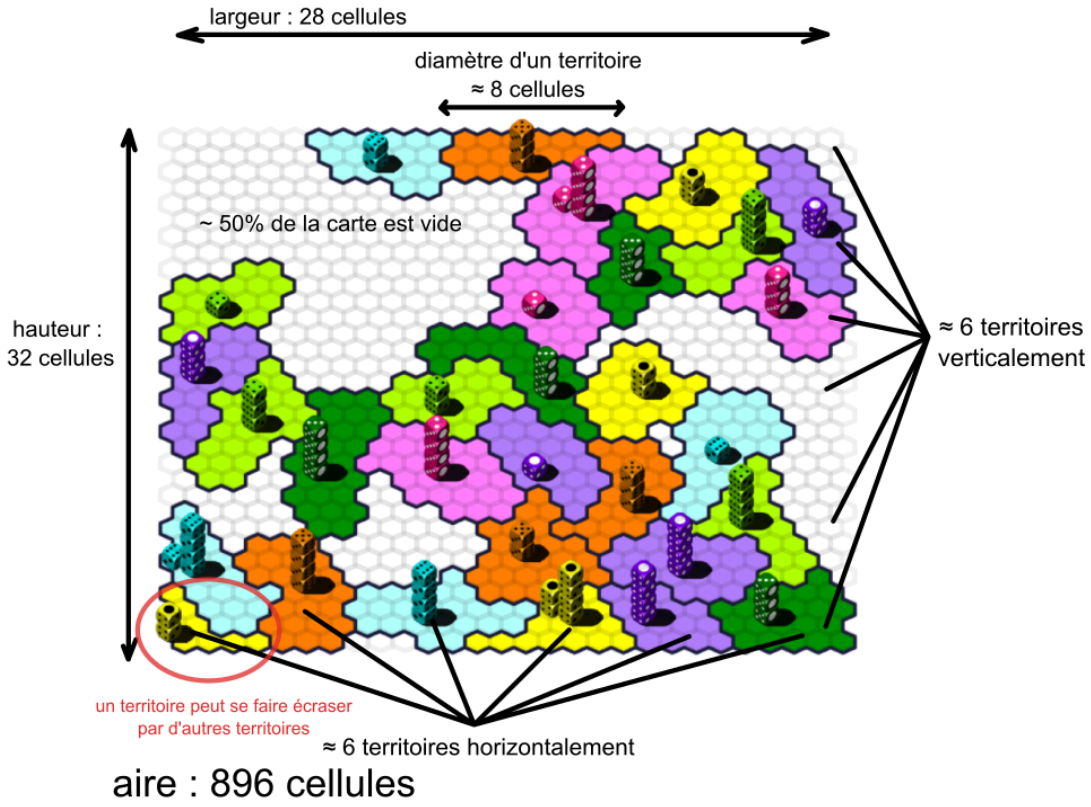
entrées : la carte, les clusters des cellules appartenant au joueur (obtenu grâce à IdentifyCluster)
sortie : un booléen : true = on peut attaquer à ce tour, false = on ne peut pas
L'algorithme va parcourir l'ensemble de la carte afin de savoir si :

- on est trop faible pour s'étendre, alors on ne va pas attaquer afin de ne pas se faire manger tout nos territoires par la suite
- on est fort mais on n'écrase pas pour autant les autres, on ne va pas attaquer car on on est fort les gens n'ont pas intérêt à nous attaquer, on va donc pouvoir gagner de plus en plus de dés pour s'étendre par la suite
- si on est "normal", extrêmement dominant ou en duel alors on a le droit d'attaquer

Génération de carte

Analyse

En premier lieu, nous avons décidé de faire une analyse sur la génération de la carte du site internet pour y déduire le maximum d'informations. On peut y tirer un certain nombre d'informations (les mesures ont étés faites sur l'application inkscape). Ci-dessous se trouve un schéma contenant l'ensemble des mesures effectuées sur une carte générée sur le site.



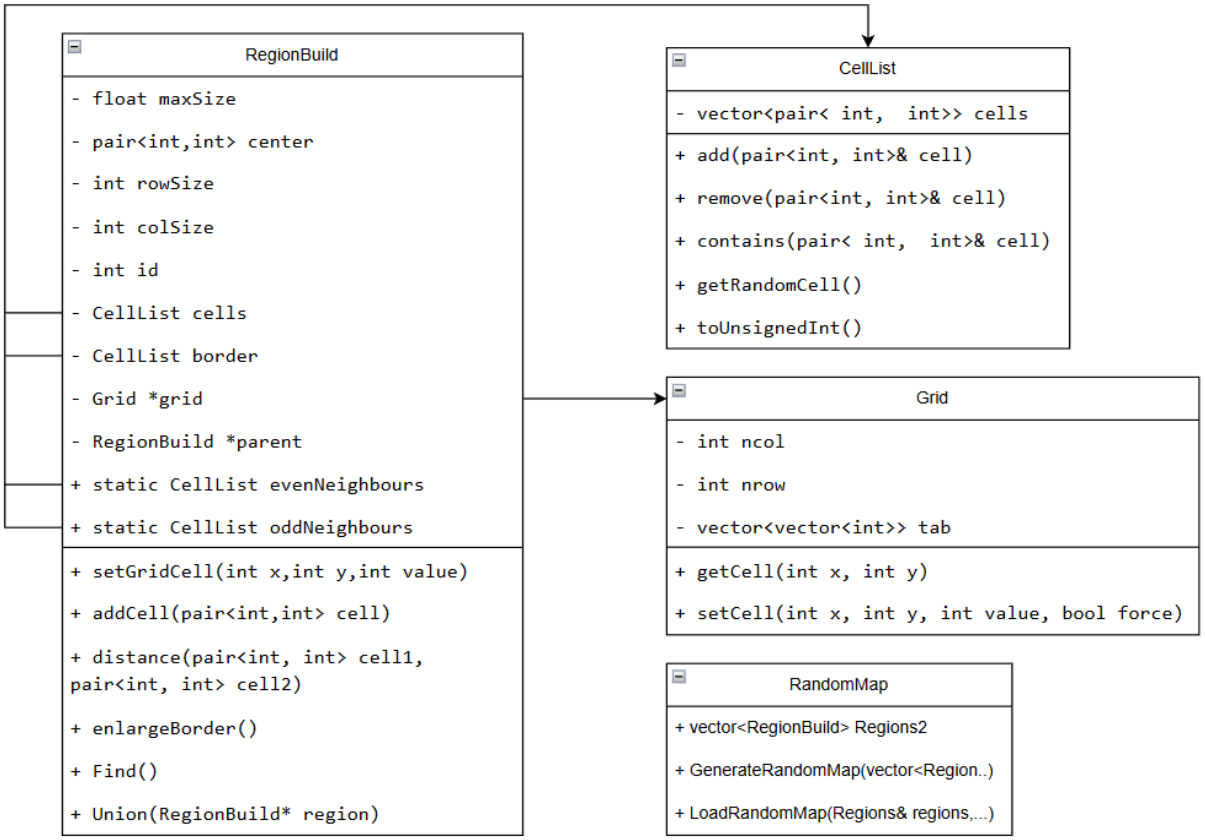
On remarque que l'on pourrait découper l'ensemble des territoires via une grille dans laquelle se trouve majoritairement un territoire de la même manière qu'on pourrait découper les départements français en une grille dont les séparations sont de la même taille. Dans l'exemple ci-dessus, on pourrait découper la carte en une grille de 6 par 6 et approximativement obtenir un territoire par cellule (en réalité la grille possèderait un peu plus de cases). Notre génération doit donc se faire en plaçant les centres des territoires aux centres de chacunes des cases de cette supposée grille.

Deux hypothèses s'offrent à nous quant à la génération de territoires vides.

La première hypothèse est que les territoires sont d'abord générés sur la grille puis une partie de ceux-ci sont retirés en fonction de leur taille ou bien d'un pourcentage de chance de vide, tout en conservant une structure connexe.

La seconde hypothèse, légèrement plus simple est que certains territoires ne sont pas du tout générés initialement, c'est celle que nous choisirons par simplicité.

Modèle Objet



Définitions et notes importantes
Cell / Cellule : hexagone, case de la grille
Region / Territoire : ensemble de cellule constituant

Pour réaliser la génération de map, nous avons utilisé la librairie std pour la majorité des structures. Celle-ci est divisée en 3 classes principales:

Grid
Grid fait référence à une grille hexagonale, la fonction setCell réalise tous les tests pour éviter les erreurs de dépassement. Une cellule ne peut également pas être écrasée par une autre, à moins de mettre l'argument force à true (utilisé lors de l'affichage).

CellList
CellList fait référence à une liste de cellules (vecteurs de pairs représentant les coordonnées x et y), dans laquelle on peut facilement ajouter, retirer des cellules mais également récupérer des cellules aléatoire via getRandomCell().

RegionBuild
RegionBuild fait référence à une région en construction.
Une structure "unir et trouver" y est implémentée afin de récupérer la plus grande composante connexe.

Toutes les RegionBuild de la génération en cours sont stockées dans region2, un vecteur de RegionBuild.

RegionBuild contient également deux variables statiques : evenNeighbours et oddNeighbours, des CellList contenant l'ensemble des coordonnées des voisins dans les cases paires et impaires.

Principe de la génération

L'algorithme de génération est entièrement original. Il à été créé afin de reproduire au mieux le résultat donné sur le site internet https://www.gamedesign.jp/games/dicewars/.

En revanche, sur le site internet, les cartes font toujours la même taille (32 par 32), dans notre cas la largeur et la hauteur de la carte peuvent varier. Deux approches se présentaient à nous : augmenter la taille des territoires en fonction de la taille de la carte ou bien augmenter le nombre de territoires selon cette dite taille. La seconde option à été choisie car nous ne voyons pas l'intérêt d'avoir simplement une résolution plus grande sur les territoires.

Les territoires auront donc toujours la même taille maximum, d'après les analyses portées sur le site un territoire possède un diamètre d'environ 8 cellules au maximum.

Le principe de génération en lui-même fonctionne comme indiqué ci-après :

La grille est divisée en plusieurs centres déterminés par le facteur de subdivision (calculé automatiquement comme la racine carrée de l'aire de la grille divisée par 3).

Les différents centres sont décalés de manière aléatoires si il y a superposition, un des centres est écrasé.

On exécute la fonction "enlargeBorder()" 35 fois sur chaque territoire (le nombre 35 à été choisi arbitrairement afin de reproduire au mieux le résultat obtenu sur le site). "enlargeBorder()" prend une cellule aléatoire de la frontière du territoire et l'ajoute à celui-ci si la cellule dépasse la taille maximale du territoire ou bien qu'elle est déjà occupée par un autre territoire ou encore qu'elle n'est pas dans la grille, elle n'est pas ajoutée.

Après ces 35 itérations, l'ensemble des territoires ont étés générés.

De part la structure chaotique et aléatoire de cet algorithme, il se peut que certains territoires soient détachés des autres, pour pallier cela, on applique l'algorithme "Unir et Trouver" afin de récupérer la composante connexe la plus grande et ajouter chacune de ses régions à la structure finale (regions).

Pour ajouter des "trous" dans la carte, un facteur d'apparition en pourcentage (fillPercent) est également pris en paramètres. A chaque apparition de centre, le pourcentage de chance que ce centre apparaisse est égale au facteur d'apparition. Dans notre cas, le pourcentage de remplissage est toujours égal à 50%. On remarque que, bien que le pourcentage de remplissage soit égal à 50%, la génération peut générer des cartes avec un pourcentage de

remplissage bien moindre (25%), cela s'explique par le fait que les régions qui ne sont pas apparues ont laissé de la place aux autres régions adjacentes.

En plus de cela, il a été ajouté des paramètres dans le cas où l'on souhaiterait générer une carte avec un nombre spécifique de territoires ("minimumRegionNumber" et "maximumRegionNumber") ainsi qu'un nombre d'essai maximum pour le faire ("maximumGenerationTry"). Cette option n'est pas activée si la valeur de "maximumGenerationTry" vaut 1 (génération par défaut). Pour cela, plusieurs instances de la carte sont générées, commençant avec un pourcentage de remplissage à 50% qui augmente au fil des essais de génération jusqu'à atteindre 100%, si les conditions de nombre de territoires sont respectées, la boucle est arrêtée et la carte est renvoyée. On notera que, bien que la complexité de l'algorithme est élevée, celui-ci n'est exécuté qu'une fois au début du code, celle-ci ne pose donc pas un grand problème et cela nous autorise à effectuer plusieurs générations.

Les paramètres sont inscrits dans le code en dur dans la fonction LoadRandomMap dans le fichier RandomMap.cpp.

Paramètres de l'algorithme

Pour récapituler, l'intégralité des paramètres de l'algorithme sont les suivants :

int r, int c
Nombre de lignes et de colonnes de la carte (la génération s'adapte automatiquement à la taille).

int maximumGénérationTry
Nombre de génération maximum dans le cas où l'on veut avoir un nombre de territoire specifique (1 sinon)

int minimumRegionNumber
Nombre de régions souhaitées au minimum dans la carte.

int maximumRegionNumber
Nombre de régions souhaitées au maximum dans la carte.

int iteration
Nombre de fois que le territoire d'une région s'étend lors de la génération

int territorySize
Rayon du territoire maximum

int offsetFactor
Décalages des centres aléatoire au début de la génération

int subdivisionFactor
Pré calculé comme étant la racine carrée de l'aire de la carte divisée par 3, nombre de fois que la carte est subdivisée lors de la génération des centres.

int fillPercent
Initialisé à 50, détermine la probabilité d'apparition d'un centre d'un territoire lors de sa création.

Conclusion

Afin de tester notre stratégie à grande échelle, nous avons développé un script (/RunAll.py) permettant d'exécuter N parties. Ce script nous permet par la suite de récupérer un ensemble de statistiques pour connaître le pourcentage de partie gagné par une stratégie en particulier.

Nous avons alors pu tester l'exécution de 5000 parties, le tout étant exécuté en 50 secondes. En moyenne, notre stratégie est au-dessus de 50% lancé dans des parties en duel face à strategie.so et au-dessus de 30% de parties gagnées lorsqu'elle est seule face à 3 strategie.so.